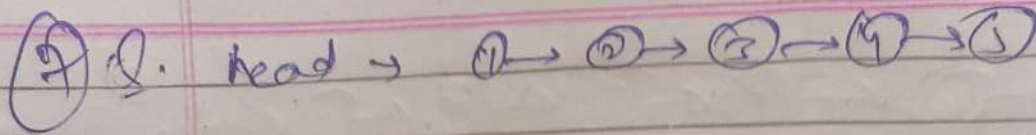


n = 2.



Step 1:- length find karo.

Step 2:- Position tak pchokocho

Step 3:- Node delete karo.

$\rightarrow$ Initially length ko zero karke.
or temp create karo go head to point
karo ga.

```
int length = 0;
```

```
ListNode temp = head;
```

```
while (temp != null) {
```

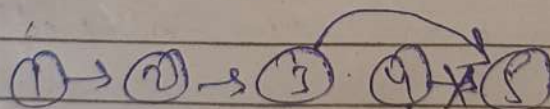
```
    temp = temp->next;
```

```
    length++;
```

```
}
```

$\rightarrow$ According to Question. Cant se. 2nd node
delete karna hai

ie



delete.

Agar ^{right} side se dekho toh 2nd node
or Agar left side se dekho toh 4th node.

To ham final position tak jana hai
length - kaha karo skip hai.

$$\text{ie length} - n = 5 - 2 + 1$$

= 4 $\rightarrow$ 4th position

from left.

$\rightarrow$ skip delete karna hai

→ Ab ek loop (circular) / target tak
poochne be liye

```
for (int i = 0; i < length - n; i++) {  
    temp = temp.next;  
}
```

~~Ab delete i.e.~~ ① → ② → ③ → ④ → ⑤.

$i = 1, i < 5 - 2 = 1 < 3, i++$

$i = 2, i < 5 - 2 = 2 < 3, i++$

$i = 3, i < 5 - 2 = 3 < 3, \text{stop.}$

matlab $i = 2$ pe hai.

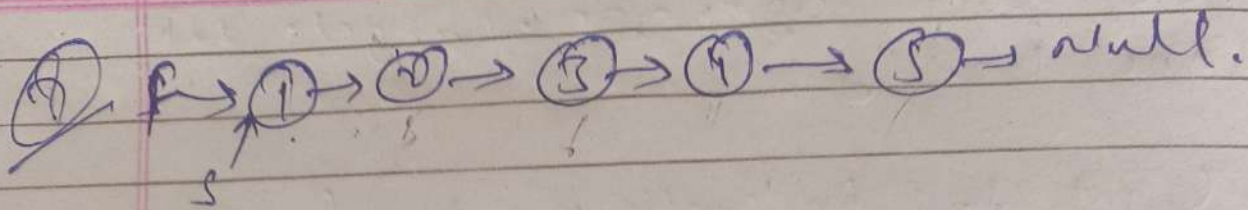
Toh temp me 3 store hoga.

→ Ab kono node delete karna hai.
agr temp me 3 store hai
→ $temp = 3 = temp.next$ (from loop).

$temp.next = temp.next.next$

At the head to return kar denge.

Another approach (slow-fast).

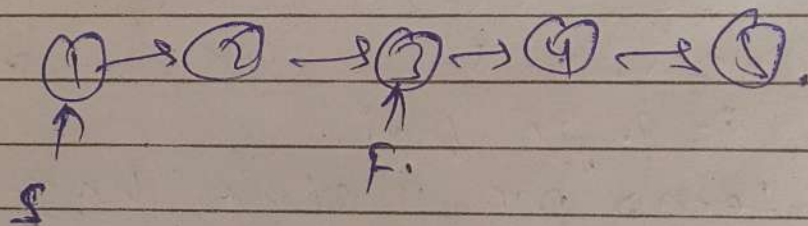


→ make ~~two~~ ^{two} pointers slow & point pointing to the head.

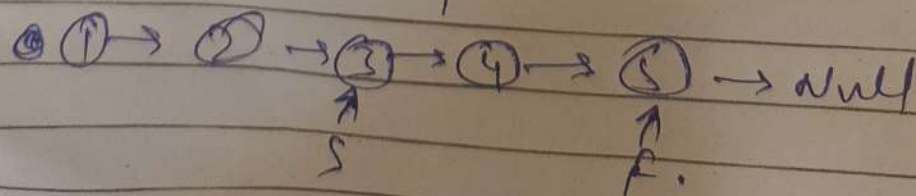
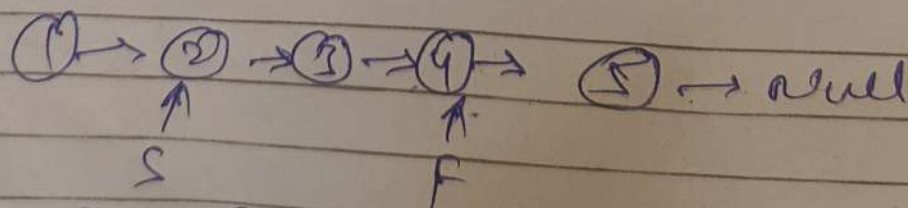
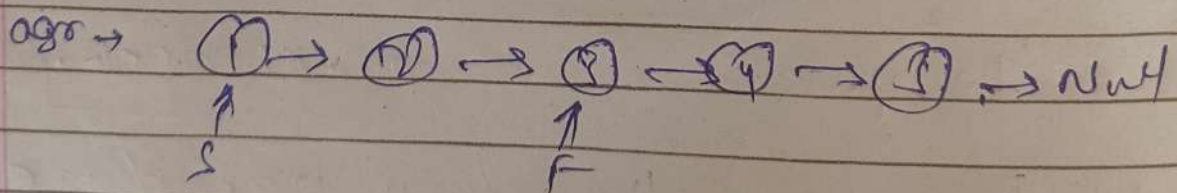
→ As fast of n step aage leke jao.

i.e. $n = 2$

then



→ Ab, dono pointer ko saath-saath aage badhaye.



→ Ab stop ~~ho~~ ho jayega kya hm loop lagayenge.

```
while (fast.next != null)
    fast = fast.next
    slow = slow.next
```

→ Iske kaon slow point jo hai wo us node ke just pehle aayega
• ~~pehle~~ is node ko delete karna hai.

→ Iske baad hm node delete ~~kar~~ karenge.

$slow.next = slow.next.next$

i.e. ① → ② → ③ → ⑤

Ans. Phir hm head ko return kar denge.

→ Ek baar ~~check~~ can hai

```
if (fast == null) {
    head.next = null;
}
```

↳ In 'head' ① → null.
or, $n = 1$.

To hme empty node return karna hai.